Circle Developer Grants · Cohort 2 | Arc mainnet

ArcBooks

Exact, reconciled and independently verifiable USDC statements for Arc, with invoice references and onchain proof.

Applicant: Mukhlis Amien, Malang, Indonesia (individual). The code is written by Claude (Anthropic) under the applicant's direction and responsibility, which is disclosed in the README.

neimasilk.github.io/arcbooks · github.com/neimasilk/arcbooks

The problem

On Arc, USDC is the native gas token (18 decimals) **and** an ERC-20 (6 decimals) over one shared balance. That is excellent for settlement and awkward for bookkeeping.

What happens onchain	What naive tools do
One ERC-20 transfer emits **two** Transfer logs (6 and 18 decimals)	Double-count the payment
A native send emits only the system log	Miss it when indexing the ERC-20
Gas emits no log	Balances silently drift
Invoice references live in a separate `Memo` contract	Ignore them

And the result is a spreadsheet from one party that proves nothing to an auditor, bank, tax consultant or counterparty.

The solution

Exact. Every USDC movement counted once, in 18-decimal integers, from the system emitter only.

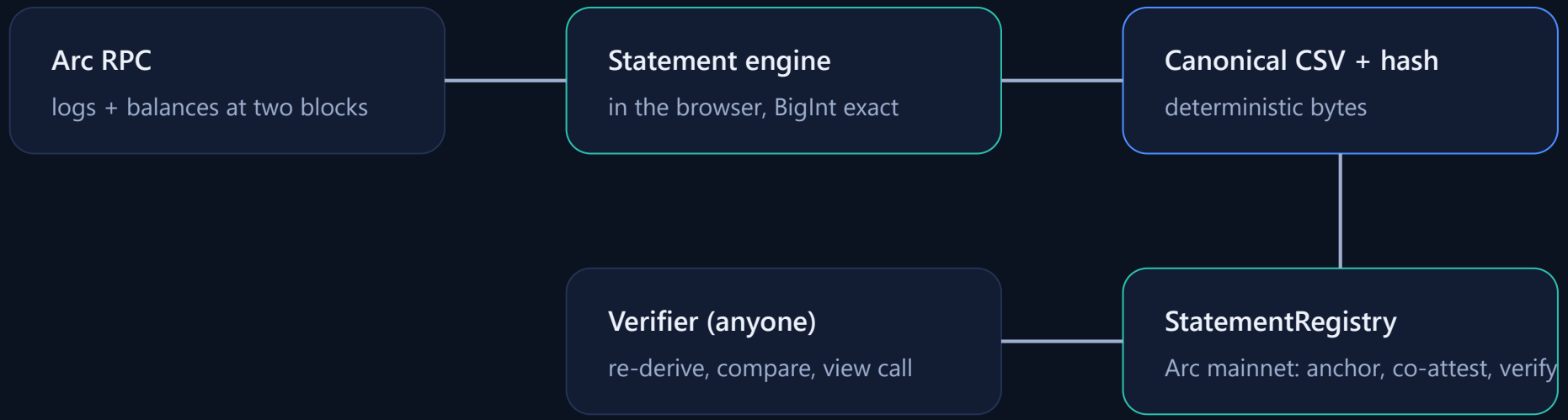
Reconciled. `opening + in - out` checked against the onchain balance. The remainder is gas, reported explicitly.

Referenced. Arc `Memo` invoice references joined to each payment.

Provable. A deterministic CSV. Its keccak256 hash is anchored in `StatementRegistry`; anyone can re-derive it from public chain data.

No backend, no indexer, no API key: the engine runs in the browser (and in Node).

How it fits together



Chainlink CRE workflow "ArcBooks Auto-Attest" rebuilds the same CSV byte for byte and writes hash-chained checkpoints on Arc Testnet.

Circle and Arc products

Integrated today

USDC: native (system-emitter logs, 18 decimals) and the ERC-20 interface. Arc's predeployed `Memo` contract for references (an Arc primitive, not a Circle developer product).

Planned in this grant

CCTP: label cross-chain arrivals. Gateway: agent micropayment accounting. Wallets: payment requests paid from developer-controlled wallets.

Priority use cases

Payments, agentic commerce (auditable books for agent wallets), and treasury/accounting for lending and capital-markets teams.

What exists today (all verifiable)

0 wei
unexplained difference on a busy mainnet account: 3,631 transfers in 3,000 blocks

2×
independent runs produce byte-identical CSVs

18
Foundry tests on the registry and attestor contracts

< 1 wk
to port the engine to another chain (MUSD Books on Mezo)

`StatementRegistry (Arc mainnet): 0xe048B7592D0aCb55f01d06013Df6961ABF9Fdb9C · block 24,009,177`

Demo merchant on mainnet: two Memo-referenced USDC payments and one native send, anchored onchain and verifiable from the app.

Traction, honestly

There are no external users yet. The product is live and verified, but there is no usage, revenue or partnership to report, and no funding to date.

- Nothing on this page counts testnet activity as traction; the CRE checkpoints are on Arc Testnet and labelled so.
- The grant funds the step from verified demo to something accountants, merchants and agent builders can rely on.
- Competition: block explorers (raw logs, no attestation), crypto accounting suites such as Cryptio or Bitwave (off-chain exports, not Arc-aware), Circle account exports (Circle accounts only). The modest moat is an open, deterministic format anyone can re-derive.

Roadmap and funding request: USD 5,000

#	Milestone (Circle product)	Success metric	Amount	Target
1	CCTP-aware statements (CCTP)	0 unexplained difference on 20+ real CCTP transfers	1,500	3 weeks
2	Agent-wallet books (Gateway, Wallets)	One agent wallet reconciled end to end, public report	2,000	6 weeks
3	Accountant pack: review link and co-attestation, multi-account, USD/IDR/EUR (USDC)	10+ statements anchored by third parties	1,500	9 weeks

The review-link flow already exists and is tested in the Mezo port; milestone 3 brings it to Arc.

Team, AI disclosure and risks

- **Solo applicant**, an individual in Indonesia, not an incorporated company.
- **AI disclosure:** the code is written by Claude (Anthropic). The applicant holds the accounts, funds deployment and is responsible for the submission. Disclosed in the README and in this deck.
- **Risk, low demand:** mitigated by payment links and the accountant co-sign flow, where users already are.
- **Risk, protocol changes:** the format is versioned and covered by live tests that fail loudly.
- **Risk, solo and AI-written code:** everything is open source, tested against live chain data and reproducible.

Links

Live app: **neimasilk.github.io/arcbooks**

Source (MIT): **github.com/neimasilk/arcbooks**

Same engine on Mezo: **github.com/neimasilk/musd-books**

Arc mainnet registry: `explorer.arc.io/address/0xe048B7592D0aCb55f01d06013Df6961ABF9Fdb9C`

Bookkeeping aid, not tax or legal advice. One account and one token per statement.